

Data Structure Lab6 : Doubly Linked List 2022-2023

Topics

1. Implement Node Class
2. Implement DoublyLinkedList Class
3. Implement Basic Methods of DoublyLinkedList
 - isEmpty()
 - size()
 - first()
 - last()
 - addFirst()
 - addLast()
 - removeFirst()
 - removeLast()

Homework

1. Describe a method for finding the middle node of a doubly linked list with header and trailer sentinels by “link hopping,” and without relying on explicit knowledge of the size of the list. In the case of an even number of nodes, report the node slightly left of center as the “middle.”

```
public class Lab6<T> { private Node<T> start; private int count; public
Lab6() { start = null; count = 0; } private static class Node<T> { T
value; Node<T> next; Node<T> prev; public Node(T value) {
this.value = value; this.next = null; this.prev = null; } }
```

2. Give an implementation of the size() method for the DoublyLinkedList class, assuming that we did not maintain size as an instance variable.

Data Structure Lab6 : Doubly Linked List 2022-2023

```
public void addAtStart(T value) { Node<T> newNode = new
Node<>(value); if (start == null) { start = newNode; start.next =
start.prev = start; } else { newNode.next = start; newNode.prev =
start.prev; start.prev.next = newNode; start.prev = newNode; start =
newNode; } count++; }
```

3. Implement the equals() method for the DoublyLinkedList class.

```
public void addAtEnd(T value) { Node<T> newNode = new
Node<>(value); if (start == null) { start = newNode; start.next =
start.prev = start; } else { newNode.next = start; newNode.prev =
start.prev; start.prev.next = newNode; start.prev = newNode; }
count++; }
```

4. Give an algorithm for concatenating two doubly linked lists L and M, with header and trailer sentinel nodes, into a single list L'.

```
public void rotateForward() { if (start != null && start.next != start) {
start = start.next; } }
```

5. Our implementation of a doubly linked list relies on two sentinel nodes, header and trailer, but a single sentinel node that guards both ends of the list should suffice. Reimplement the DoublyLinkedList class using only one sentinel node.

```
public void rotateBackward() { if (start != null && start.prev != start) {
start = start.prev; } }
```

6. Implement a circular version of a doubly linked list, without any sentinels, that supports all the public behaviors of the original as well as two new update methods, rotate() and rotateBackward.

```
public void removeFromStart() { if (start == null) return; if (start.next
== start) { start = null; } else { start.next.prev = start.prev;
start.prev.next = start.next; start = start.next; } count--; } public void
removeFromEnd() { if (start == null) return; if (start.next == start) {
```

Data Structure Lab6 : Doubly Linked List 2022-2023

```
start = null; } else { start.prev.prev.next = start; start.prev =
start.prev.prev; } count--; } public void displayList() { if (start == null)
return; Node<T> current = start; do { System.out.print(current.value +
" "); current = current.next; } while (current != start);
System.out.println(); }
```

7. Implement the clone() method for the DoublyLinkedList class.

```
public Lab6<T> duplicate() {
    Lab6<T> newList = new Lab6<>();
    if (start == null) return newList;

    Node<T> current = start;
    do {
        newList.addAtEnd(current.value);
        current = current.next;
    } while (current != start);

    return newList;
}
```